

Dependently Typed World Models

Toward Proof-Carrying Learned Simulators

Anonymous

Submitted for review

August 2026

Abstract

World models—learned simulators of environment dynamics used in model-based reinforcement learning—have demonstrated strong empirical performance yet remain difficult to endow with machine-checked invariants. We argue that dependent type theory, and in particular the algebraic structures made tractable in the Rocq proof assistant, provides a natural semantic framework for strengthening world models with verifiable guarantees. We introduce the notion of a *proof-carrying world model*: a learned simulator whose latent states, observation operators, and planning procedures are annotated with machine-checked witnesses of admissibility. We show how partial commutative monoids (PCMs) over finitely keyed maps—already developed for separation-logic reasoning about heap-like resources—can be reinterpreted as a compositional semantics for heterogeneous agent histories, cached beliefs, and capability scopes. We develop small illustrative Rocq formalizations of these ideas and sketch a research agenda toward alignment-aware planning under typed resource constraints.

Keywords: world models, dependent type theory, Rocq, partial commutative monoids, model-based reinforcement learning, proof-carrying programs, alignment.

1. Introduction

The success of model-based reinforcement learning (MBRL) over the past decade rests on a single insight: an agent that can simulate its environment can plan efficiently without exhausting the real environment in trial [Sutton, 1990, Ha and Schmidhuber, 2018, Hafner et al., 2020, Schrittwieser et al., 2020]. The learned simulator—often called a *world model*—compresses sensory history into a compact latent state, predicts the consequences of candidate actions, and thereby enables forward search at a fraction of the sample cost required by purely reactive approaches [Hafner et al., 2023, Ye et al., 2021].

Despite this empirical progress, world models share a structural limitation with most learned systems: their internal representations carry no machine-verifiable account of what they are permitted to represent or what invariants they are committed to preserving. Admissibility conditions, scope constraints, and capability limits are expressed, when they are expressed at all, in natural language documentation, external monitors, or post-hoc audits. They are, in the language of software engineering, *extrinsic* to the model rather than *intrinsic* to it.

This paper argues that dependent type theory provides the missing vocabulary. In a dependently typed setting, an object may carry, as part of its very type, a machine-checked proof that it satisfies the conditions required for its use. Proof need not arrive as a chaperone after the fact; it can accompany the value from the moment of construction. We draw in particular on the Rocq proof assistant [The Coq/Rocq Development Team, 2024] and on the FCSL-PCM library [Nanevski et al., 2014], which develops partial commutative monoids (PCMs) as a clean algebraic model of separable, composable resources.

Our contributions are:

1. A conceptual framework—the *proof-carrying world model*—that articulates how dependent types can annotate the operational interfaces of a learned simulator (§3).
2. An account of how PCMs, originally developed for separation-logic reasoning over heap memory, can be reinterpreted as a compositional semantics for agent observation histories and capability scopes (§4).
3. Small Rocq formalizations illustrating these ideas, from basic proof-carrying state through typed rollout traces to heap-style ownership of sensor boundaries (§5).
4. A discussion of research directions, including connections to alignment-aware planning and multi-agent trust (§6).

We do not claim a complete implementation. We offer a research direction and a working vocabulary for future development.

2. Background

2.1. World Models in Reinforcement Learning

A world model, in the MBRL sense, is a parametric function $f_\theta : \mathcal{Z} \times \mathcal{A} \rightarrow \mathcal{Z} \times \mathcal{R}$ mapping a latent state $z \in \mathcal{Z}$ and action $a \in \mathcal{A}$ to a predicted successor state

and reward signal. Practical systems augment this with encoder and decoder networks that map between observation space and latent space, and with planning modules—cross-entropy method [Botev et al., 2013], Monte Carlo Tree Search [Coulom, 2006], or learned value functions [Hafner et al., 2023]—that search over imagined trajectories.

Key empirical milestones include Dyna [Sutton, 1990], which first demonstrated the efficiency of planning inside a learned model; World Models [Ha and Schmidhuber, 2018], which popularized recurrent variational world models for visual tasks; DreamerV1–V3 [Hafner et al., 2020, 2021, 2023], which showed that long-horizon imagination can match or exceed model-free baselines; and MuZero [Schrittwieser et al., 2020], which learned a model sufficient for planning without requiring faithful reconstruction of observations.

A recurring theme is the tension between *predictive accuracy* and *structural fidelity*. Predictively accurate models may still fail to capture the constraints—resource limits, scope boundaries, capability revocations—that govern what a system is permitted to do, as distinct from what it expects to happen. This gap is the starting point for our proposal.

2.2. Dependent Type Theory and the Rocq Proof Assistant

Dependent type theory [Martin-Löf, 1984, Coquand and Huet, 1988] extends the Curry–Howard correspondence [Howard, 1980, Wadler, 2015] so that types may contain, and depend upon, ordinary program values. A type such as $\text{Vec}(n)$ —a vector of exactly n elements—encodes a length invariant that the type-checker enforces at compile time, without separate runtime assertion. More generally, a value of type $\{x : A \mid P(x)\}$ is both a value in A and a proof of $P(x)$, inseparably.

Rocq (formerly Coq) [The Coq/Rocq Development Team, 2024] implements the Calculus of Inductive Constructions [Coquand and Huet, 1988, Paulin-Mohring, 2015], a rich dependent type theory supporting:

- *Inductive types* and pattern matching;
- *Dependent records* (**Record**) that bundle values with their correctness conditions;
- *Proof terms* as first-class citizens of the same language as programs;
- *Extraction* to OCaml or Haskell, enabling certified programs to run outside the proof environment.

The MathComp library [Gonthier et al., 2013] and the FCSL-PCM library [Nanevski et al., 2014] provide mature algebraic foundations—including Boolean reflection, finite maps, and partial commutative monoids—on which we build.

2.3. Partial Commutative Monoids

Definition 2.1 (Partial Commutative Monoid). *A partial commutative monoid (PCM) is a set U equipped with a distinguished element $\mathbf{0} \in U$, a validity predicate*

$\text{valid} : U \rightarrow \text{Prop}$, and a partial binary operation $\oplus : U \rightarrow U \rightarrow U$ satisfying:

$$\begin{array}{ll} \text{commutativity :} & x \oplus y = y \oplus x \\ \text{associativity :} & (x \oplus y) \oplus z = x \oplus (y \oplus z) \\ \text{unit :} & x \oplus \mathbf{0} = x \\ \text{validity domain :} & \text{valid}(x \oplus y) \Rightarrow \text{valid}(x) \wedge \text{valid}(y) \end{array}$$

When $\text{valid}(x \oplus y)$ fails, $x \oplus y$ returns a distinguished undefined element Undef .

PCMs arose in the context of separation logic [Reynolds, 2002, O’Hearn, 2019] as the canonical semantic domain for *separating conjunction*: two resources x and y may be combined if and only if their join is valid—that is, they do not overlap or conflict. The FCSL-PCM library [Nanevski et al., 2014] provides a rich Rocq hierarchy of PCM instances, including heap-like finite maps, which we repurpose in §4.

3. A Framework for Proof-Carrying World Models

3.1. Motivation

The latent state z of a world model typically has no intrinsic type other than “tensor of floats.” Its admissibility—whether it encodes a state the system is permitted to reason about—is enforced, if at all, by external wrappers, evaluation callbacks, or human review. We propose to make admissibility *intrinsic*: the type of a latent state should include evidence of its own legitimacy.

More concretely, consider a multi-agent planning system in which different sub-models govern different capability scopes—one model has read access to sensor S_1 , another controls actuator A_3 . In a conventional architecture, scope enforcement is handled by runtime checks, policy tables, or application-level guards that sit outside the model itself. A dependently typed architecture would express scope as part of the type of the state: the type of “state with actuator A_3 capability” literally differs from the type of “state without it.” Scope confusion becomes a type error, detected before execution.

3.2. Proof-Carrying State

We define a *proof-carrying state* as a dependent record bundling a representational value with a machine-checked admissibility witness.

Definition 3.1 (Proof-Carrying State). *Let S be a set of representational states and let $\text{Adm} : S \rightarrow \text{Prop}$ be an admissibility predicate. A proof-carrying state is a pair (s, h) where $s \in S$ and $h : \text{Adm}(s)$.*

The key property is that proof-carrying states *cannot be constructed in an inadmissible configuration*: construction of the pair requires producing the proof h simultaneously with the value s . There is no window between construction and certification.

3.3. Proof-Carrying World Model Interface

A world model operating over proof-carrying states must be designed so that:

1. *Observation closure*: applying the observation operator to an admissible state produces an admissible state.
2. *Imagination validity*: applying the imagination (rollout) operator to an admissible state produces a valid (non-undefined) state.

These are not runtime checks but proof obligations discharged at definition time. A world model that cannot satisfy them is, in a precise sense, *ill-typed*—it cannot be instantiated.

The following Rocq sketch captures the interface:

```
From mathcomp Require Import ssreflect ssrbool ssrnat seq.
From pcm Require Import pcm.

Section TypedWorld.
Context {State : pcm}.

Record WorldModel := {
  observe   : State -> seq nat -> State;
  imagine   : State -> seq nat -> State;
  admissible : State -> Prop;
  observe_ok : forall s frame,
    admissible s -> admissible (observe s frame);
  imagine_ok : forall s actions,
    admissible s -> valid (imagine s actions)
}.

End TypedWorld.
```

Listing 1: Proof-carrying world model interface in Rocq

The `State` is parameterized by an arbitrary PCM, so the framework is not committed to any particular representation; heap-like finite maps, trace histories, and richer algebraic structures are all valid instantiations.

4. PCMs as Compositional Semantics for Agent Histories

4.1. From Heap Reasoning to History Composition

Separation logic was designed to reason locally: a Hoare triple in separation logic is valid for a fragment of the heap, and the frame rule extends it to arbitrarily larger heaps that the procedure does not touch [Reynolds, 2002]. The key semantic enabler is that heap composition via $\oplus$ is defined only when the constituent fragments are disjoint—capturing the intuition that two independent computations cannot silently share state.

We propose an analogous discipline for world model histories. An *observation history* is a finite map from timesteps to observation records; a *capability scope*

is a finite map from capability identifiers to authorization witnesses. Two such fragments can be joined if and only if their key sets are disjoint—that is, if they record disjoint portions of the agent’s experience or authority.

Definition 4.1 (History PCM). *Let $\mathcal{T}$ be a countable set of timesteps and $\mathcal{O}$ a set of observation records. Define $\mathcal{H} = \mathcal{T} \multimap \mathcal{O}$ (partial functions with finite support). The PCM structure on $\mathcal{H}$ is:*

$$\begin{aligned} \text{valid}(h) &\triangleq h \neq \text{Undef} \\ h_1 \oplus h_2 &\triangleq \begin{cases} h_1 \cup h_2 & \text{if } \text{dom}(h_1) \cap \text{dom}(h_2) = \emptyset \\ \text{Undef} & \text{otherwise} \end{cases} \end{aligned}$$

This is exactly the PCM of disjoint-key finite maps, realized in Rocq by `natmap` [Nanevski et al., 2014].

Under this reading, combining two histories is only valid if they record distinct moments in time. Conflicting entries—two agents writing different observations for the same timestep—produce an undefined result, signaling a representational conflict that the planner must not silently resolve.

4.2. Typed Rollout Traces

A *rollout trace* in our framework is a proof-carrying history: a finite map from timesteps to rewards, annotated with a positive-horizon witness. The following Rocq definition makes this concrete:

```
From mathcomp Require Import ssreflect ssrbool ssrnat seq.
From pcm Require Import pcm natmap.

Record rollout := {
  trace      : history nat;
  horizon    : nat;
  horizon_pos : horizon > 0
}.

Definition extend_trace (r : rollout) (t reward : nat) :=
  if t == 0 then None
  else if valid ((t \-> reward) \+ trace r)
    then Some {| trace      := (t \-> reward) \+ trace
                  r;
                  horizon    := horizon r;
                  horizon_pos := horizon_pos r |}
    else None.
```

Listing 2: Typed rollout trace

The `extend_trace` function is partial: it returns `None` whenever the new timestep is zero (a sentinel for invalid time) or whenever the new entry conflicts with an existing entry in the map. This partiality is not an error condition to be caught at runtime; it is the honest signature of an operation that may legitimately fail to produce a valid result.

4.3. Capability Scopes as Resource Types

Beyond observation histories, PCMs provide a natural model for *capability scopes*—the set of actions an agent is currently authorized to perform. We represent a capability scope as a finite map from capability identifiers to authorization tokens (represented, for simplicity, as natural numbers). Two scopes may be combined only if they govern disjoint sets of capabilities. An agent attempting to act under a capability it does not hold faces a type error at the point of action construction, not a runtime permission denial.

This design separates two concerns that are often conflated in practice: *what an agent expects to happen* (the predictive content of the world model) and *what an agent is permitted to cause* (the normative content, now encoded in types). Violations of the latter are detected constructively, before any real-world action is taken.

5. Illustrative Rocq Formalizations

5.1. Proof-Carrying State: A Minimal Example

Before introducing PCM structure, we illustrate the simplest instance of proof-carrying state: a rollout record whose `horizon` field is certified by a length witness.

```
From Coq Require Import List Arith.
Import ListNotations.

Record CheckedRollout := {
  horizon          : nat;
  actions          : list nat;
  horizon_matches  : length actions = horizon
}.

Definition single_action (a : nat) : CheckedRollout :=
{| horizon          := 1;
  actions          := [a];
  horizon_matches  := eq_refl |}.

```

Listing 3: Minimal proof-carrying rollout record

The field `horizon_matches` is not a runtime assertion but a proof term. It is discharged by `eq_refl` at the point of construction, confirming that the singleton list `[a]` has length 1. A `CheckedRollout` with mismatched horizon cannot be constructed; the attempt would produce a type error.

5.2. Sensor Ownership via Heap-Style PCMs

We illustrate resource-typed sensor access using the `heap` PCM from FCSL-PCM. A sensor write is defined to return `Undef` if the target pointer is null, and a lemma certifies that non-null writes preserve validity.

```

From mathcomp Require Import ssreflect ssrbool.
From pcm Require Import heap.

Definition sensor_write (p : ptr) (v : dynamic id) (h :
  heap) :=
  if p == null then Undef else upd p v h.

Lemma sensor_write_preserves_nonnull p v h :
  p != null ->
    sensor_write p v h != Undef.
Proof.
move=> p_nonnull.
rewrite /sensor_write p_nonnull.
case: h=> // = fm pf.
by case: decP=> // /eqP.
Qed.

```

Listing 4: Heap-style ownership for sensor boundary

The lemma `sensor_write_preserves_nonnull` is a machine-checked guarantee. Any system that calls `sensor_write` on a non-null pointer inherits the proof; any downstream code that requires a valid heap fragment can accept the result directly.

5.3. Composing Agent Histories

The following sketch demonstrates how two disjoint history fragments may be safely combined into a single rollout.

```

From mathcomp Require Import ssreflect ssrbool ssrnat.
From pcm Require Import pcm natmap.

(* Two disjoint observation fragments *)
Definition obs1 : history nat := 1 \-> 42.
Definition obs2 : history nat := 2 \-> 17.

(* Their join is valid because key sets are disjoint *)
Lemma obs_join_valid : valid (obs1 \+ obs2).
Proof. by rewrite /obs1 /obs2; compute. Qed.

(* An attempt to join conflicting entries produces Undef *)
Definition obs1' : history nat := 1 \-> 99.

Lemma obs_conflict_invalid : ~~ valid (obs1 \+ obs1').
Proof. by rewrite /obs1 /obs1'; compute. Qed.

```

Listing 5: Safe composition of disjoint observation fragments

These two lemmas capture the core safety property: valid composition is not assumed or checked at runtime; it is established once, by proof, and then inherited by any code that depends on the composed history.

6. Discussion and Research Directions

6.1. Alignment-Aware Planning

The alignment problem for learned agents [Russell, 2019, Gabriel, 2020] is, in part, a problem of specification: how does an agent know that the actions it is considering are within the scope of what it is supposed to do? Current approaches rely heavily on reward shaping, RLHF [Christiano et al., 2017, Bai et al., 2022b], and constitutional constraints [Bai et al., 2022a], all of which are intrinsically extrinsic—they surround the model with evaluative signals rather than embedding constraints within its representational structure.

A proof-carrying world model offers a complementary approach. By requiring that high-stakes action types carry typed witnesses of their preconditions, the planning procedure is structurally prevented from producing plans that would violate stated constraints—not because a guard intercepts the output, but because the output type cannot be constructed without the required evidence. The difference is analogous to the difference between a runtime bounds check and a statically verified array access: the latter eliminates a class of error by construction, before execution begins.

We envision a planning module whose action type is parameterized by a capability scope κ : an action of type **Action**(κ) may only exercise the capabilities enumerated in κ . The planner’s type signature then enforces that it cannot produce an action requiring capability c unless $c \in \kappa$. Scope escalation becomes a type error.

6.2. Multi-Agent Trust and Fragment Provenance

In multi-agent settings, observations arrive from diverse sources with varying degrees of trust. A dependently typed world model can represent this by annotating observation histories with *provenance types*: a history of type **Hist**(τ) was collected under trust level τ , and the type system can enforce that a planner operating under trust level τ' may only act on histories whose provenance satisfies $\tau' \preceq \tau$ for some trust ordering $\preceq$.

The PCM structure ensures that joining histories of incompatible provenance produces an **Undef** result rather than a silently untrusted composite. This makes cross-provenance contamination detectable at the type level—a property that has practical significance wherever agents interact with adversarially constructed inputs or aggregated data of heterogeneous origin.

6.3. Connections to Verified Machine Learning

Our proposal is broadly consistent with a growing literature on *verified machine learning* [Seshia et al., 2018, Katz et al., 2019, Liu et al., 2021] and *certified programs* [Leroy, 2009, Klein et al., 2009]. These works pursue machine-checked correctness for specific properties—robustness bounds, absence of undefined behavior—but typically treat the neural component as a black box to be analyzed from the outside. Our contribution is to argue that the *interface* of a world model—its observation operators, imagination rollouts, and planning predicates—can itself be given a dependently typed specification that interoperates naturally with proof-assistant toolchains.

This is complementary to, rather than in competition with, techniques for verifying the *internals* of neural networks. A world model whose interface carries machine-checked guarantees can be composed with a certifying planner even if the neural weights themselves remain opaque. The typed interface becomes the contract between the learned and the verified parts of the system.

6.4. Toward a Full Formal Development

The formalizations in §5 are illustrative sketches, not a complete verified system. A full development would require:

1. A Rocq formalization of the world model interface parameterized by an arbitrary PCM, with proofs of the stated closure properties.
2. A typed rollout algebra: a Rocq structure capturing the sequential composition of proof-carrying transitions under horizon constraints.
3. A formalization of capability scopes as PCMs over finite maps of authorization tokens, with lifting lemmas connecting scope composition to the planning type.
4. An extraction pipeline enabling certified planning code to run inside a standard MBRL training loop.

We leave this development as future work, noting that each component has precedent: the FCSL-PCM library provides the algebraic foundations; verified functional programs with extraction are routine in Rocq; and typed planning interfaces have been explored in the context of verified task planners [Blanchette and Nipkow, 2010].

7. Related Work

World models. Ha and Schmidhuber’s World Models [Ha and Schmidhuber, 2018] established the “dream-then-act” paradigm for visual planning. The DreamerV1–V3 series [Hafner et al., 2020, 2021, 2023] demonstrated that continuous control and Atari tasks can be mastered through pure imagination. MuZero [Schrittwieser et al., 2020] showed that a latent model without ground-truth reconstruction suffices for search-based planning. EfficientZero [Ye et al., 2021] extended this to extreme sample efficiency. None of these architectures address typed invariants on the model interface.

Formal methods and learning. Seshia et al. [Seshia et al., 2018] survey formal specification and verification techniques applicable to learning-enabled systems, emphasizing runtime monitors and assume-guarantee contracts. Neural network verification [Katz et al., 2019, Liu et al., 2021] focuses on robustness of fixed networks. CompCert [Leroy, 2009] and seL4 [Klein et al., 2009] demonstrate that large software systems can be given machine-checked correctness proofs; our work proposes that learned interfaces can participate in such developments.

Separation logic and resources. Reynolds [Reynolds, 2002] introduced separation logic; O’Hearn [O’Hearn, 2019] surveys two decades of development. Nanevski et al. [Nanevski et al., 2014] develop FCSL as a Hoare-logic framework for concurrent separation reasoning, with a mature Rocq library. We repurpose this infrastructure for compositional history reasoning.

Alignment and specification. Russell [Russell, 2019] frames alignment as a problem of misspecified objectives. Christiano et al. [Christiano et al., 2017] and Bai et al. [Bai et al., 2022b,a] address alignment through human feedback and constitutional constraints. Gabriel [Gabriel, 2020] provides a conceptual taxonomy. Our proposal is orthogonal: rather than specifying objectives more accurately, we propose to specify the operational interface of the model more precisely.

8. Conclusion

We have argued that the latent states, observation operators, and planning procedures of learned world models are appropriate targets for dependent type annotation, and that partial commutative monoids provide a natural algebraic language for the compositional reasoning this requires. The resulting notion of a *proof-carrying world model* shifts certain safety and scope guarantees from runtime monitors and external policy tables to the type structure of the model itself—making violations detectable before execution rather than after.

The proposal is not a complete solution to the deep challenges of learned-agent reliability. A world model whose interface is well-typed may still learn an inaccurate model of environment dynamics; proof-carrying state does not substitute for empirical validity. What it does is ensure that the commitments the system makes about scope, provenance, and composition are expressed in a form that can be checked mechanically, composed reliably, and depended upon by downstream verified components.

In that modest but precise sense, dependent type theory offers machine learning something it has rarely been asked to accept: not a new performance benchmark, but an obligation to say clearly what it knows—and to carry the proof.

References

Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022a.

- Yuntao Bai et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022b.
- Jasmin Christian Blanchette and Tobias Nipkow. Nitpick: A counterexample generator for higher-order logic based on a relational model finder. In *Interactive Theorem Proving (ITP)*, pages 131–146, 2010.
- Zdravko I. Botev, Dirk P. Kroese, Reuven Y. Rubinstein, and Pierre L’Ecuyer. The cross-entropy method for optimization. *Handbook of Statistics*, 31:35–59, 2013.
- Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Thierry Coquand and Gérard Huet. The calculus of constructions. volume 76, pages 95–120, 1988.
- Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *Computers and Games*, pages 72–83, 2006.
- Iason Gabriel. Artificial intelligence, values, and alignment. *Minds and Machines*, 30:411–437, 2020.
- Georges Gonthier et al. A machine-checked proof of the odd order theorem. pages 163–179, 2013.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.
- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations (ICLR)*, 2021.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- William A. Howard. The formulae-as-types notion of construction. In Jonathan P. Seldin and J. Roger Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus, and Formalism*, pages 479–490. Academic Press, 1980.
- Guy Katz, Derek A. Huang, Duligur Ibeling, et al. The marabou framework for verification and analysis of deep neural networks. In *Computer Aided Verification (CAV)*, pages 443–452, 2019.
- Gerwin Klein et al. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- Changliu Liu, Tomer Arnon, Christopher Lazarus, Christopher Strong, Clark Barrett, and Mykel J. Kochenderfer. Algorithms for verifying deep neural networks. volume 4, pages 244–404, 2021.
- Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- Aleksandar Nanevski, Ruy Ley-Wild, Ilya Sergey, and Germán Andrés Morales. Communicating state transition systems for fine-grained concurrent resources. In *European Symposium on Programming (ESOP)*, 2014. FCSL-PCM library: <https://github.com/imdea-software/fcsl-pcm>.
- Peter W. O’Hearn. Separation logic. *Communications of the ACM*, 62(2):86–95, 2019.
- Christine Paulin-Mohring. Introduction to the calculus of inductive constructions. In David Delahaye and Bruno Woltzenlogel Paleo, editors, *All about Proofs, Proofs for All*. College Publications, 2015.
- John C. Reynolds. Separation logic: A logic for shared mutable data structures. pages 55–74, 2002.
- Stuart Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering atari, go, chess and shogi by planning with a learned model. In *Nature*, volume 588, pages 604–609, 2020.
- Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. Formal specification for deep neural networks. pages 20–34, 2018.
- Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. pages 216–224, 1990.
- The Coq/Rocq Development Team. The rocq proof assistant reference manual. <https://rocq-prover.org/>, 2024.
- Philip Wadler. Propositions as types. *Communications of the ACM*, 58(12):75–84, 2015.
- Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.